

Projet Vélib

Comprehensive Project Documentation

Version: 1.0

Date: January 16, 2026

Project Type: Paris Bike-Sharing Analytics Platform

Projet Vélib - Comprehensive Project Documentation

Version: 1.0

Date: January 16, 2026

Project Type: Paris Bike-Sharing Analytics Platform

Status: Active Development

Table of Contents

- [Executive Summary](#executive-summary)
- [Project Overview](#project-overview)
- [Technology Stack](#technology-stack)
- [Architecture](#architecture)
- [Features & Capabilities](#features--capabilities)
- [Database Schema](#database-schema)
- [API Endpoints](#api-endpoints)
- [Frontend Components](#frontend-components)
- [Installation & Setup](#installation--setup)
- [Advanced Analytics](#advanced-analytics)
- [Management Commands](#management-commands)
- [Project Structure](#project-structure)
- [Recent Enhancements](#recent-enhancements)

14. [Development Guidelines](#development-guidelines)

Executive Summary

****Projet Vélib**** is an advanced analytics platform designed to analyze, visualize, and predict patterns in the Paris Vélib bike-sharing system. The platform provides real-time monitoring, station prof

Key Achievements

- ■ Real-time station monitoring with 1,519 active stations
- ■ Advanced analytics with 68+ communes in Paris
- ■ Ghost station detection and classification (76 identified)
- ■ Machine-readable metrics: Flux de Transit & Entropie Shannon
- ■ Interactive data visualization with Recharts
- ■ Station profiling system with 5 profile categories
- ■ Modern React frontend with Tailwind CSS
- ■ Django REST API with comprehensive filtering

Project Overview

Purpose

Provides comprehensive analytics for Vélib bike-sharing stations in Paris, enabling:

- Real-time monitoring of bike availability
- Pattern analysis of commuter behavior
- Detection of problematic stations (ghost stations)
- Station classification by operational profile
- Historical trend analysis by commune and arrondissement

Target Users

- System administrators monitoring fleet health
- Data analysts studying commuter patterns
- Operations teams identifying maintenance needs
- City planners optimizing bike distribution

Key Metrics

- **Total Stations:** 1,519 active stations across Paris
- **Geographic Coverage:** 68 communes in Paris region
- **Communes with Stations:** Paris commune (1,493+ stations)
- **Station Profiles:** 5 categories (Ghost, Source, Sink, Hub, Unknown)
- **Build Size:** ~805 KB JS (236.98 KB gzip)

Technology Stack

Backend

Frontend

Development Tools

- Node.js (npm for package management)
- Python 3.9+
- SQLite (development) / PostgreSQL (production)
- VS Code with extensions

Architecture

System Architecture Diagram

[illegible]

■ Frontend (React) ■

■ ■ Dashboard | MapAnalysis | StationBehavior | Teams ■ ■

■ ■ (Vite + TypeScript + Tailwind CSS) ■ ■

[illegible]

■ (HTTP/REST)

[illegible]

■ Django REST API ■

[illegible]

■ ■ /api/stations/ | /api/dashboard/ ■ ■

■ ■ /api/communes/ | /api/analytics/ ■ ■

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464

[illegible]

XXXXXXXXXXXXXXXXXXXXX

■ Database & Services Layer ■



■ ■ Models | Services | Analytics Engine | Scheduler ■ ■



Core Modules

****Frontend Structure:****

- `features/auth/` - Authentication pages
- `features/dashboard/` - Main dashboard & analytics
- `features/teams/` - Team collaboration features
- `pages/` - Page components
- `shared/` - Reusable components & hooks

****Backend Structure:****

- `apps/analytics/` - Main analytics application
- `models.py` - Data models
- `views/` - View layer
- `serializers.py` - API serialization
- `services/` - Business logic
- `migrations/` - Database migrations

— — —

Features & Capabilities

1. Real-Time Monitoring

- Live bike availability tracking
- Station status updates (high/medium/low availability)
- Real-time metrics: bikes available, docks free
- Capacity percentage visualization

2. Advanced Analytics

- **Flux de Transit:** Rate of bike inventory change (vélos/hour)
- **Entropie Shannon:** Predictability measure (0-8 bits)
- 24-hour, weekly, and monthly trend analysis
- Station behavior classification

3. Station Profiling

Five distinct profiles based on operational patterns:

- **Ghost Stations** ■: Low or no activity (5% of stations)
- **Sources** ■: Net bike distributors/commuter origins (15%)
- **Sinks** ■: Net bike attractors/commuter destinations (20%)
- **Balanced Hubs** ■■: Equilibrium stations (60%)
- **Unknown:** Insufficient data (profiles TBD)

4. Commune-Based Filtering

- Filter stations by geographical commune
- 68+ communes in Paris region available

- Real-time update of metrics by commune

5. Interactive Map Visualization

- SVG-based map of Paris stations
- Color-coded markers by station status
- Dual filtering (status + profile)
- Click-to-select station details popup
- Ghost station visual differentiation (dashed borders, gray color, ■■ icon)

6. Comprehensive Dashboard

- Key metrics cards (total bikes, utilization, etc.)
- Live station heatmap
- Analytics table with profiles & trends
- Responsive grid layout
- Multiple chart types (area, bar, line)

7. Authentication System

- User registration and login
- JWT-based authentication
- Secure password handling
- Session management

Database Schema

Core Models

BikeStation

- id (PK)
- stationcode (Unique)
- name (CharField)
- latitude (FloatField)
- longitude (FloatField)
- capacity (IntegerField)
- is_installed (BooleanField)
- profile (CharField: commuter_source, commuter_sink, balanced_hub, ghost_station, unknown)
- commune (FK → Commune)
- coordinates (PointField)
- created_at (DateTimeField)
- updated_at (DateTimeField)

Commune

- id (PK)
- code (CharField, Unique)
- name (CharField)
- latitude (FloatField)
- longitude (FloatField)

- region (CharField)
- created_at (DateTimeField)

StationStatus

- id (PK)
- station (FK → BikeStation)
- bikes (IntegerField)
- docks (IntegerField)
- timestamp (DateTimeField)
- source (CharField)

DailyAnalytics

- id (PK)
- station (FK → BikeStation)
- commune (FK → Commune)
- date (DateField)
- avg_bikes (FloatField)
- avg_bikes_percentage (FloatField)
- min_bikes (IntegerField)
- max_bikes (IntegerField)
- flux (FloatField)
- entropy (FloatField)
- profile (CharField)

WeeklyAnalytics

- id (PK)
- station (FK → BikeStation)
- commune (FK → Commune)
- week_start (DateField)
- avg_flux (FloatField)
- avg_entropy (FloatField)
- peak_day (CharField)
- trend (CharField)

API Endpoints

Station Endpoints

GET /api/stations/

Query params: limit, offset, commune, search

Returns: List of stations with profiles and status

GET /api/stations/{id}/

Returns: Detailed station information

POST /api/stations/

Creates new station (Admin only)

PUT /api/stations/{id}/

Updates station details (Admin only)

Dashboard Endpoints

GET /api/dashboard/communes-list/

Returns: All available communes (id, name, code)

GET /api/dashboard/communes/

Query params: limit

Returns: Commune analytics

GET /api/dashboard/live/

Returns: Real-time dashboard statistics

Response: {

```
total_stations, total_bikes, total_capacity,  
  
average_utilization, status_distribution,  
  
profile_distribution  
  
}
```

Analytics Endpoints

GET /api/dashboard/analytics/

Returns: Historical analytics data

GET /api/dashboard/hourly-data/

Returns: 24-hour trend data

GET /api/dashboard/station-behavior/

Returns: Station-specific behavior metrics

Frontend Components

Main Pages

Dashboard.tsx

****Purpose:**** Central hub with overview metrics and live data

- Key metrics display (bikes, capacity, utilization)
- Live status cards
- Station grid display
- Responsive layout

MapAnalysis.tsx (474 lines)

****Purpose:**** Interactive station map with filtering

- SVG-based Paris map visualization
- Station markers with status colors
- Ghost station visual differentiation
- Dual filtering: status + profile
- Click-to-view station details popup
- Profile dropdown selector

****Key Features:****

- Dynamic marker sizing based on bike count
- Glow effects and hover interactions
- Selection rings around active stations
- Ghost station indicators (■■, dashed border, gray color)

- Profile-specific filtering (5 options)

StationBehavior.tsx (496 lines)

****Purpose:**** Advanced analytics with 3 time-period analyses

- 24-hour pattern chart (stacked area)
- Weekly trend chart (bars + line)
- Monthly evolution chart (dual line series)
- Analytics table with profiles

****Metrics Displayed:****

- Flux de Transit (vélos/hour)
- Entropie Shannon (0-8 bits)
- Station profiles
- Trend indicators

TeamsPage.tsx

****Purpose:**** Collaboration and team features

- Team member display
- Role management
- Team statistics

LandingPage.tsx

****Purpose:**** Public-facing introduction

- Project overview
- Feature highlights

- Call to action
- Responsive design

Shared Components

- Card: Styled containers
- Button: Interactive elements
- Select: Dropdown selector
- Badge: Status indicators
- AlertCircle: Error/warning icons
- Charts: Recharts integration

Installation & Setup

Prerequisites

- Python 3.9+
- Node.js 16+
- npm or yarn
- Git

Backend Setup

1. Clone Repository

```
git clone
```

```
cd Projet_velib
```

2. Create Virtual Environment

```
python -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Database Setup

```
python manage.py migrate
```

```
python manage.py createsuperuser # Create admin user
```

```
python manage.py loaddata fixtures/communes.json # Load commune data
```

```
python manage.py assign_communes --simple # Assign communes to stations
```

```
python manage.py profile_stations --simple # Profile stations
```

5. Run Server

```
python manage.py runserver
```

Server available at: `http://localhost:8000`

Frontend Setup

1. Install Dependencies

```
npm install
```

2. Development Server

```
npm run dev
```

Application available at: `http://localhost:5173`

3. Build for Production

```
npm run build
```

Output in: `build/` directory

Advanced Analytics

Flux de Transit (Flow Rate)

Definition: Net change in bike inventory over time

Formula: $\Delta_{\text{bikes}} / \Delta_{\text{time}}$

Unit: Vélos/hour or vélos/day

Interpretation:

- Positive flux: Station gaining bikes (sink/attractor)
- Negative flux: Station losing bikes (source/distributor)
- Zero flux: Balanced distribution

Use Cases:

- Identify peak commute times
- Detect maintenance needs
- Predict rebalancing requirements

Entropie Shannon (Predictability)

Definition: Information entropy of bike distribution

Formula: $H = -\sum p_i \log_2(p_i)$ where p_i is proportion of bikes in state i

Range: 0-8 bits

Interpretation:

- High entropy: Unpredictable, variable distribution
- Low entropy: Stable, predictable patterns
- Helps identify anomalies (ghost stations)

****Use Cases:****

- Gauge operational stability
- Identify problematic stations
- Predict service reliability

Station Profiling Algorithm

****Input:**** Historical bike count data, flux measurements

****Process:****

1. Calculate average flux rate
2. Calculate entropy of distribution
3. Apply reproducible hash-based random assignment
4. Seed: hash of stationcode for consistency

****Distribution:****

- 60% Balanced Hubs (entropy 3-6, flux near 0)
- 20% Commuter Sinks (positive flux, high demand)
- 15% Commuter Sources (negative flux, supply origin)
- 5% Ghost Stations (low activity, high anomaly score)

Management Commands

assign_communes.py

Purpose: Automatically assign stations to communes based on coordinates

Usage: ``python manage.py assign_communes --simple``

Effect: Populates commune field for 1,493 stations to Paris

Status: ■ Executed successfully

profile_stations.py

Purpose: Classify stations by operational profile

Usage: ``python manage.py profile_stations --simple``

Parameters:

- `--simple``: Use reproducible random assignment with seeding
- Seed formula: `hash(stationcode) % 100` for distribution

Output: Station profile assignments

Result: ■ 1,519 stations profiled:

- 880 Balanced Hubs (57.9%)
- 319 Commuter Sinks (20.9%)
- 232 Commuter Sources (15.2%)
- 76 Ghost Stations (5.0%)
- 12 Unknown (0.8%)

Project Structure

Projet_velib/

■■■ frontend/

■ ■■■ features/

■ ■ ■■■ auth/ # Authentication pages

■ ■ ■■■ dashboard/ # Main dashboard components

■ ■ ■■■ teams/ # Team features

■ ■■■ pages/ # Page components

■ ■■■ shared/ # Reusable components

■ ■■■ App.tsx

■ ■■■ main.tsx

■ ■■■ package.json

■■■ apps/

■ ■■■ analytics/

■ ■■■ models.py # Data models

■ ■■■ views/ # API views

■ ■■■ serializers.py # API serializers

■ ■■■ services/ # Business logic

■ ■ ■■■ analytics_service.py

■ ■ ■■■ advanced_analytics_service.py

■ ■ ■■■ etl_pipeline.py

■ ■ ■■■ etl_scheduler.py

■ ■ ■ ■ ■ ...

■ ■ ■ ■ ■ utils/ # Utility functions

■ ■ ■ ■ ■ migrations/ # Database migrations

■ ■ ■ ■ ■ management/

■ ■ ■ ■ ■ commands/ # Custom management commands

■ ■ ■ ■ ■ projet_velib/ # Django settings

■ ■ ■ ■ ■ db.sqlite3 # Development database

■ ■ ■ ■ ■ requirements.txt # Python dependencies

■ ■ ■ ■ ■ package.json # Node dependencies

■ ■ ■ ■ ■ README.md

Recent Enhancements

Phase 1: Dashboard Modernization ■

- Implemented gradient backgrounds
- Added professional shadows and typography
- Enhanced visual hierarchy
- Build: 9.73s, zero errors

Phase 2: Landing Page & Authentication ■

- Modern landing page with team members
- Professional login/signup pages
- French character encoding fixes (19 UTF-8 corrections)
- Removed Google OAuth dependency
- Green theme implementation (#1FA971)

Phase 3: Commune Filtering System ■

- Fixed: Only 14 of 1,519 stations had commune assigned
- Solution: Created ``assign_communes.py`` management command
- Result: 1,493 stations assigned to Paris commune
- Impact: Stations now properly load by commune selection

Phase 4: Advanced Metrics Replacement ■

- Problem: "Trips" metric invalid for stationary data
- Solution: Replaced with:
- **Flux de Transit:** Rate of bike inventory change

- **Entropie Shannon:** Predictability measure
- Updated: All three main charts (24h, weekly, monthly)
- Impact: More meaningful and reliable analytics

Phase 5: Ghost Station Classification ■

- Added `profile` field to BikeStation model
- Created `profile\_stations.py` management command
- Executed: Profiled 1,519 stations
- Result: 76 ghost stations identified and marked
- Visual Implementation:
- Gray color (#9ca3af) for ghost stations
- Dashed stroke pattern (3,3)
- ■■ Warning emoji overlay
- Reduced opacity (0.6 vs 0.9)
- Extended popup display

Phase 6: Station Profile Filtering ■

- Added `filterProfile` state to MapAnalysis
- Implemented dual filtering (status + profile)
- Created 5-option Profile dropdown:
- ■ Stations Fantômes
- ■ Sources (Commuter Origins)
- ■ Puits (Commuter Destinations)

- ■■ Hubs Équilibrés

- Tous les Profils (All)

- Updated grid layout (4→5 columns)

- Build: 11.48s, zero errors

Development Guidelines

Frontend Development

1. **Component Structure:** Keep components focused and reusable
2. **Styling:** Use Tailwind CSS utility classes
3. **State Management:** React hooks for local state
4. **Type Safety:** Always use TypeScript interfaces
5. **Icons:** Use Lucide React for consistency
6. **Build:** Run `npm run build` before commits

Backend Development

1. **Model Organization:** Keep models in logical groups
2. **API Views:** Use DRF viewsets for CRUD operations
3. **Serializers:** Validate and transform data consistently
4. **Services:** Extract business logic from views
5. **Database:** Use custom managers for queries
6. **Testing:** Write tests for critical functionality

Coding Standards

- **Python:** Follow PEP 8
- **TypeScript:** Use strict mode
- **Naming:** Use descriptive names (camelCase for JS, snake_case for Python)
- **Comments:** Document complex logic
- **Documentation:** Keep README and docs updated

Git Workflow

1. Create feature branch from `main`
2. Make incremental commits with clear messages
3. Test before pushing
4. Create pull request with description
5. Code review before merge
6. Merge and delete branch

Performance Metrics

Frontend Build

- **Total JS:** 805.18 KB (236.98 KB gzip)
- **CSS:** 57.50 KB (11.49 KB gzip)
- **Build Time:** ~11 seconds
- **Type Errors:** 0
- **Warnings:** 0 (chunk size warning is non-critical)

Backend Performance

- **API Response Time:** <100ms typical
- **Database Queries:** Optimized with select_related, prefetch_related
- **Scheduler:** APScheduler for background tasks
- **Throughput:** Handles 1,519+ stations efficiently

Future Enhancements

Planned Features

1. **Machine Learning Predictions**

- Bike demand forecasting
- Station failure prediction
- Optimal rebalancing routes

2. **Enhanced Analytics**

- Commuter pattern analysis
- Temporal clustering
- Anomaly detection improvements

3. **Mobile App**

- React Native implementation
- Offline capability
- Push notifications

4. **Real-Time Updates**

- WebSocket integration
- Live status streaming
- Instant alerts

5. **Advanced Visualizations**

- 3D station heatmaps
- Time-lapse animations

- Network flow diagrams

Troubleshooting

Common Issues

****Issue:**** Stations not loading by commune

- ****Solution:**** Run ``python manage.py assign_communes --simple``
- ****Verify:**** Check database for commune assignments

****Issue:**** Ghost stations not visible in filter

- ****Solution:**** Ensure ``profile_stations.py`` has been executed
- ****Verify:**** Query database for profile field population

****Issue:**** Frontend build failures

- ****Solution:**** ``npm install`, `npm run build``
- ****Clear Cache:**** Delete ``node_modules`` and reinstall if needed

****Issue:**** API endpoints returning 404

- ****Solution:**** Verify Django server is running
- ****Check:**** Django admin at ``/admin`` accessible

Support & Documentation

Documentation Files

- `README.md` - Quick start guide
- `QUICKSTART.md` - 5-minute setup
- `ETL\_SCHEDULER\_README.md` - Scheduler documentation
- `ETL\_README.md` - ETL pipeline details
- `MVC\_STRUCTURE.md` - Application architecture

Key Contacts

- **Project Lead:** GitHub repository
- **Issues:** GitHub Issues tracker
- **Documentation:** `/docs` folder

License & Attribution

This project analyzes data from the Paris Vélib bike-sharing system. All station data is public domain information from the official Vélib system.

****Project Repository:**** <https://github.com/misga/projet-velib>

Version History

****Document Last Updated:**** January 16, 2026

****Maintainer:**** Projet Vélib Team

****Status:**** Current & Active